

Dynamics and Forces Documentation

Authors: Geenadie Rathnayake, Mak Sukimoto, Kyle Wittcoff

2025-04-14

Contents

I. Nomenclature	2
II. Seaglider Dynamics and Forces Model	3
II.A. Assumptions	4
II.B. Coordinate Systems	4
II.C. Inputs: state and control vectors	5
II.D. Coordinate Transformations	7
II.E. Flow Angles	8
II.F. Hydrodynamic Forces	8
II.G. Buoyancy and Gravity	10
II.H. Mass and Inertia	11
II.I. Battery Motion	12
II.J. CG Offset	12
II.K. Moments	12
II.L. Kinematics	13
II.M. Equations of Motion	14
II.N. State Propagation	16
III. Sub-system Requirements	18
IV. Error Budget Allocation for Dynamics and Force Requirements	19
IV.A. Error Percent Allocation for States (used in PDR and CDR)	19
IV.B. Error Magnitude Calculations (used in TRR, AR)	21
V. References	22
VI. Appendix A: Tables and Figures	23
VII. Appendix B: Final Code (dynamicsv7.m)	26

I. Nomenclature

Table 1: Definition of Variables

Symbol	Description	Symbol	Description
x	Horizontal position	α	Angle of attack
y	Lateral position	γ	Glide slope angle
z	Vertical position	β	Sideslip angle
θ	Pitch angle	L	Lift force
ϕ	Roll angle	D	Drag force
ψ	Yaw angle	B	Buoyant force
Desired heading	Target heading with respect to magnetic north	Y	Side force
Heading	Actual or current heading of the glider	$\mathbf{F}_w$	Force vector in wind frame
$\mathbf{v}_b$	Translational velocity vector in body frame	$\mathbf{F}_{aero}$	Force vector in body frame
V	Translational speed	T_p	Roll torque
$\mathbf{v}_g$	Translational velocity vector in glide-path frame	T_q	Pitch torque
$\mathbf{\Omega}$	Angular velocity vector in body frame	T_r	Yaw torque
$\mathbf{P}_p$	Linear momentum of m_{bat} in body frame	$\mathbf{T}_{hydro}$	Torque vector in body frame
$\mathbf{M}$	Total mass matrix, $\mathbf{M} = m_s \mathbf{I} + \mathbf{M}_f$	S	Glider reference area in the XY plane
$\mathbf{M}_f$	Added mass matrix	$\bar{c}$	Mean aerodynamic chord (MAC)
m_s	Stationary glider mass	b	Wing span
m_{bat}	Mass of battery pack	T	Temperature ($^{\circ}\text{C}$)
m_{total}	Total glider mass, $m_s + m_{bat}$	T_0	Ambient surface temperature ($^{\circ}\text{C}$)
m_0	Net mass, $m_s + m_{bat} - m_{disp}$	P	Pressure (dbar)
$\mathbf{J}$	Total inertia matrix, $\mathbf{J} = \mathbf{J}_s + \mathbf{J}_f$	Salt	Salinity
$\mathbf{J}_f$	Added inertia matrix	ρ	Density, $\rho(P, T, \text{Salt})$
$\mathbf{J}_s$	Inertia matrix of stationary mass	κ	Compressibility factor of the seaglider
$\mathbf{r}_p$	Position vector of m_{bat} in body frame	τ	Volumetric expansion coefficient of the seaglider
g	Gravitational acceleration	Vol_{static}	Seaglider volume without VBD
q	Dynamic pressure	Vol_{blad}	Volume of oil in bladder
$\mathbf{r}_s$	Position vector of CG of static mass	Vol_{disp}	Displaced volume, $Vol_{static} + Vol_{blad}$

II. Seaglider Dynamics and Forces Model

Following design is reflected in the `dynamicsV7.m` file.

Dynamics Modeler Design Overview

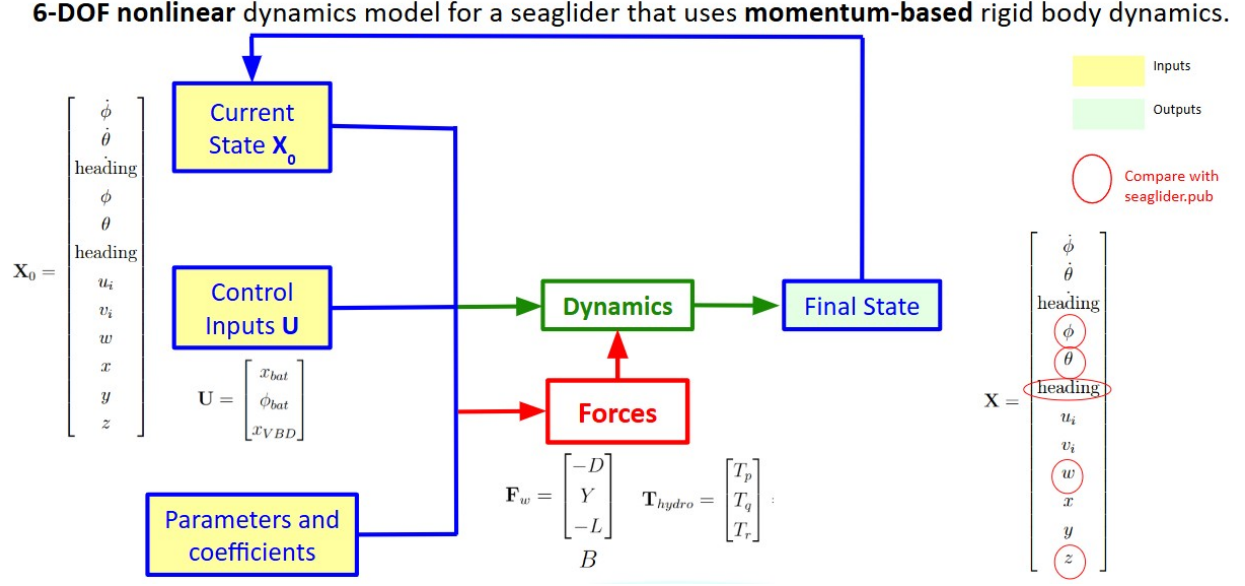


Fig. 1: Dynamics Block Diagram

Figure 1 shows inputs and outputs of the 6-DOF nonlinear dynamics model used to simulate the motion of the seaglider. The model takes the current state, $\mathbf{X}_0$, as an input, which includes the vehicle's angular rates, Euler angles, translational velocities WRT heading, and inertial position. It also uses the control input vector, $\mathbf{U}$, which includes the battery position, battery rotation angle, and variable buoyancy device input. The parameters and coefficients block provides the physical properties of the glider, such as mass, inertia, geometry, added mass, and hydrodynamic coefficients. The forces block calculates the external forces and moments acting on the Seaglider, including drag, side force, lift, and hydrodynamic moments. These inputs are passed into the dynamics block, which uses momentum based equations of motion to calculate the output which is the next state, $\mathbf{X}$, of the glider. The final state which contains the same state variables as the initial state, becomes the starting point for the next time step.

Forces Modeler Design Overview

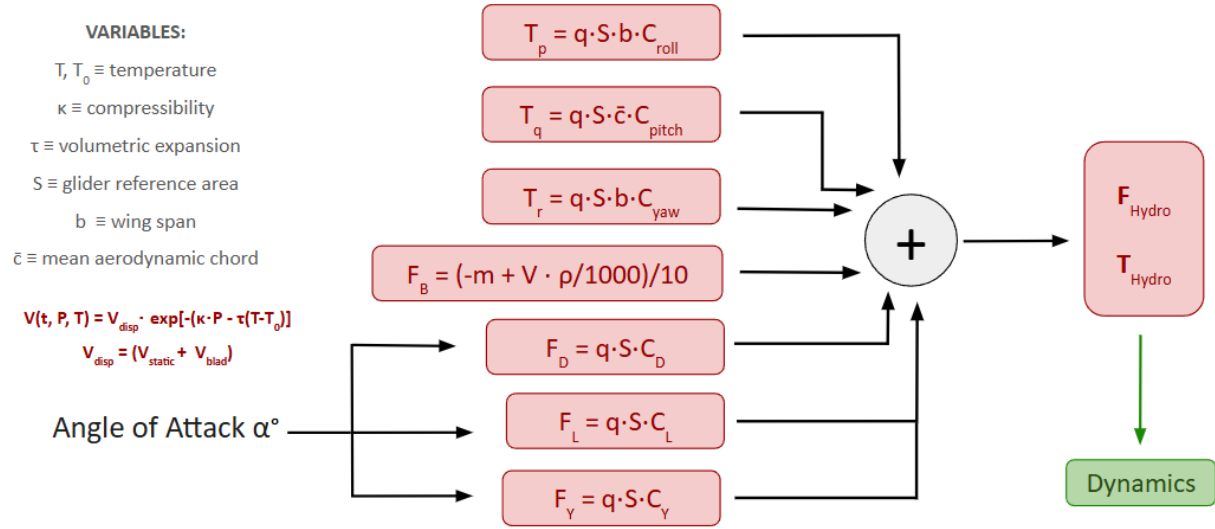


Fig. 2: Enter Caption

Figure 2 shows the flow of the force block of the plant. The inputs to the plant include density, volume, angle of attack, dynamic pressure, the coefficients of pitch, roll, yaw moments, and the coefficients of lift, drag, and side forces. These inputs are used to calculate the lift, drag, and side forces as well as roll, yaw, and pitch moments. The output of the plant are two vectors of the hydrodynamic forces (lift, drag, side, and buoyancy forces) and hydrodynamic torques (pitch, roll, yaw moments). The final output for the forces model is input into the dynamics modeler which then outputs the final state vector.

II.A. Assumptions

#	Assumption	Impact
1	Actuation of the VBD does not affect the location of the CB	Low, possibly moderate
2	Added mass and inertia from the wings is negligible	Moderate, possibly high

II.B. Coordinate Systems

Two coordinate frames are directly used in this function: an inertial frame referred to as the "Glide Path Frame" and a non-inertial, body frame. The "Glide Path Frame" is the Earth centered frame rotated about its vertical axis by the desired heading. This was chosen over

a North-East-Down, Earth centered inertial frame for its ease in tracking the Seaglider's progress down its desired dive path. This frame is static, as the desired heading does not change throughout a dive.

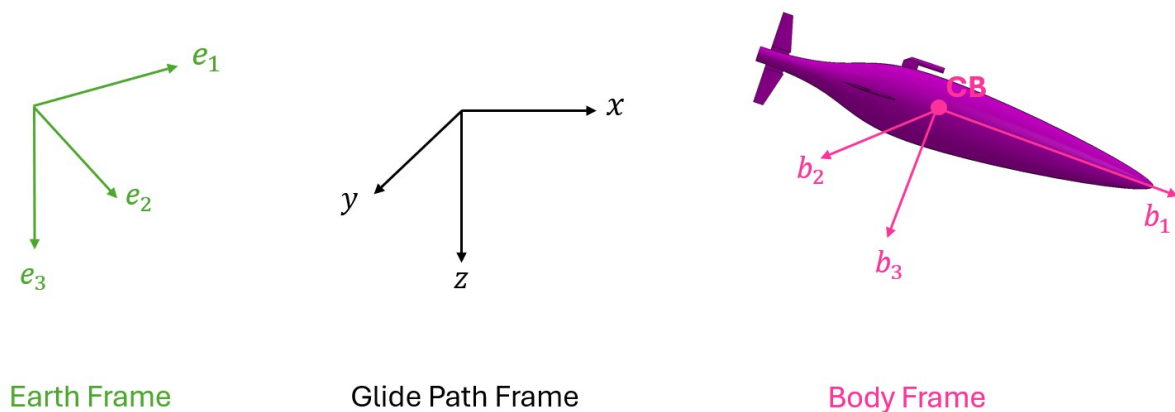


Fig. 3: Three coordinate systems used in dynamics

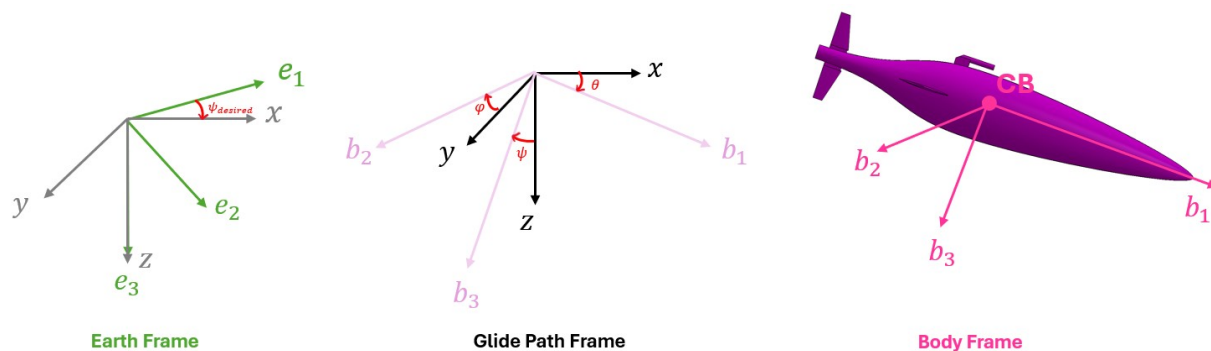


Fig. 4: Coordinate Transformations

II.C. Inputs: state and control vectors

The seaglider is modeled as a nonlinear 6-DOF system with state vector:

$$\mathbf{X} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \text{heading} \\ \phi \\ \theta \\ \text{heading} \\ u_i \\ v_i \\ w \\ x \\ y \\ z \end{bmatrix} \quad (1)$$

where,

- $\phi, \theta, \text{heading}$: the three Euler angles measured by the attitude sensor
- $\dot{\phi}, \dot{\theta}, \text{heading}$: the roll rate, pitch rate, and heading rate in body frame
- u_i, v_i, w : translational velocities WRT the current heading
- x, y, z : positions in the glide-path frame

The initial state vector is the state vector at the beginning of the dive which is taken in as an input by the dynamics code at the beginning of the dive.

The heading in the Earth frame is translated to ψ in the glide path frame by

$$\psi = \text{heading} - \text{desired heading} \quad (2)$$

The horizontal and lateral velocity WRT the heading, u_i and v_i , are translated to the glide path frame by:

$$u = u_i \cos(\psi) - v_i \sin(\psi) \quad \text{and} \quad v = u_i \sin(\psi) + v_i \cos(\psi) \quad (3)$$

In vector form this is:

$$R = \begin{bmatrix} \cos(\psi) & -\sin(\psi) \\ \sin(\psi) & \cos(\psi) \end{bmatrix} \quad (4)$$

When calculating acceleration, one must use the chain rule and the derivative of R . This derivative $\dot{R}$ is:

$$\dot{R} = \dot{\psi} \begin{bmatrix} -\sin(\psi) & -\cos(\psi) \\ \cos(\psi) & -\sin(\psi) \end{bmatrix} \quad (5)$$

The next input taken in by the dynamics is the control vector. The control input vector is defined as:

$$\mathbf{U} = \begin{bmatrix} x_{bat} \\ \phi_{bat} \\ VBD_{ctl,cc} \end{bmatrix} \quad (6)$$

where,

- x_{bat} : battery position along b_1 -axis in body frame
- ϕ_{bat} : battery rotation angle in the yz -plane
- $V_{VBD,cc}$: volume of oil in Variant Buoyancy Device (VBD) in units of cubic centimeters

II.D. Coordinate Transformations

The transformation between the body and glide-path frames is defined using a direction cosine matrix (DCM) where `angle2dcm` function is used in `Matlab` to obtain $\mathbf{DCM}_{bg}$:

$$\mathbf{DCM}_{bg} = \mathbf{DCM}(\psi, \theta, \phi) \quad \mathbf{DCM}_{gb} = \mathbf{DCM}_{bg}^T \quad (7)$$

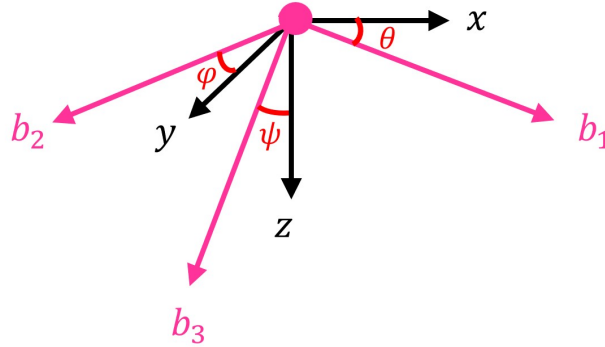


Fig. 5: Coordinate transformation from body frame to glide path frame.

The velocity in the body frame is:

$$\mathbf{v}_b = \mathbf{DCM}_{gb} \mathbf{v}_g = \begin{bmatrix} u_b \\ v_b \\ w_b \end{bmatrix}, \quad \text{where} \quad \mathbf{v}_g = \begin{bmatrix} u \\ v \\ w \end{bmatrix} \quad (8)$$

II.E. Flow Angles

The angle of attack α and sideslip angle β are given by:

$$\alpha = \tan^{-1} \left(\frac{w_b}{u_b} \right) \quad (9)$$

$$\beta = \sin^{-1} \left(\frac{v_b}{V} \right) \quad (10)$$

where the speed of the glider is:

$$V = \sqrt{u^2 + v^2 + w^2} \quad (11)$$

The glide angle in glide path frame is given by (reference charlies paper):

$$\gamma = \tan^{-1} \left(\frac{w}{u} \right) \quad (12)$$

The relationship between glide angle and angle of attack is given by (reference chalie's paper):

$$\alpha = \gamma - \theta \quad (13)$$

II.F. Hydrodynamic Forces

The dynamic pressure is defined as:

$$q = \frac{1}{2} \rho V^2 \quad (14)$$

Where $\rho = \rho(P, T, S)$ and is taken directly from the Seaglider .nc files.

Force coefficients are obtained from CFD and wind tunnel testing:

$$C_L = f_L(\alpha, \beta), \quad C_D = f_D(\alpha, \beta), \quad C_Y = f_Y(\alpha, \beta) \quad (15)$$

where C_L is the lift coefficient, C_D is the drag coefficient, and C_Y is the side force coefficient.

These coefficients are found at a series of M angles of attack (α) and N sideslip angles (β). These are compiled into an $M \times N$ matrix to be double interpolated as a lookup-table.

The forces in the body frame are:

$$\mathbf{F}_{hydro} = \begin{bmatrix} -D \\ Y \\ -L \end{bmatrix} = \begin{bmatrix} -qSC_D \\ qSC_Y \\ -qSC_L \end{bmatrix} \quad (16)$$

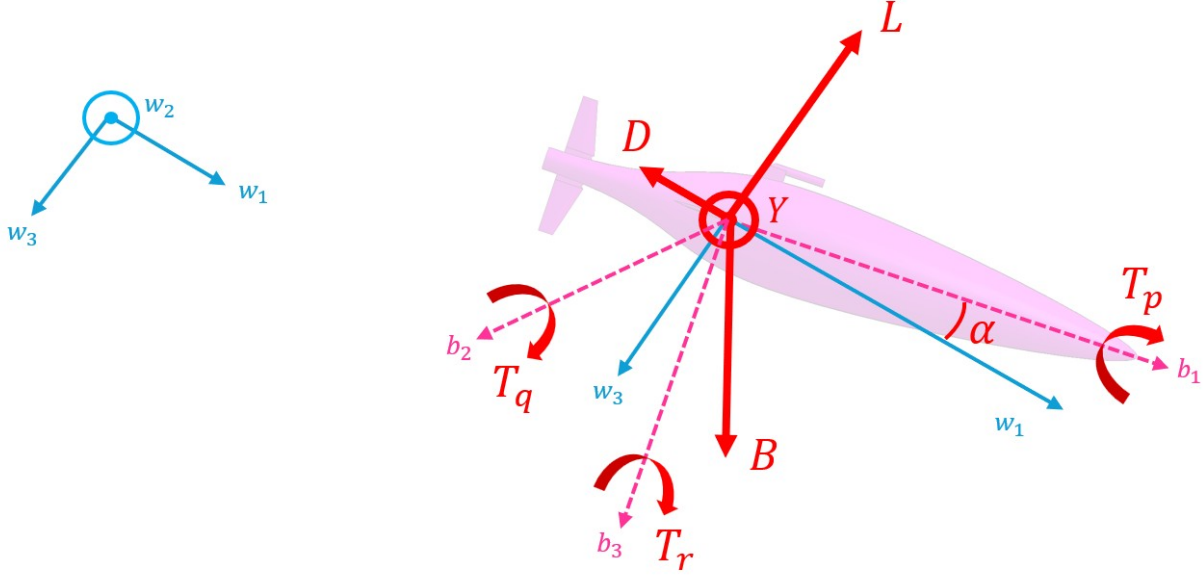


Fig. 6: Free Body Diagram

The transformation between the body and wind frames is defined using a direction cosine matrix (DCM) where `angle2dcm` function is used in `Matlab` to obtain $\mathbf{DCM}_{bw}$:

$$\mathbf{DCM}_{bw} = \mathbf{DCM}(\alpha, \beta) \quad \mathbf{DCM}_{wb} = \mathbf{DCM}_{bw}^T \quad (17)$$

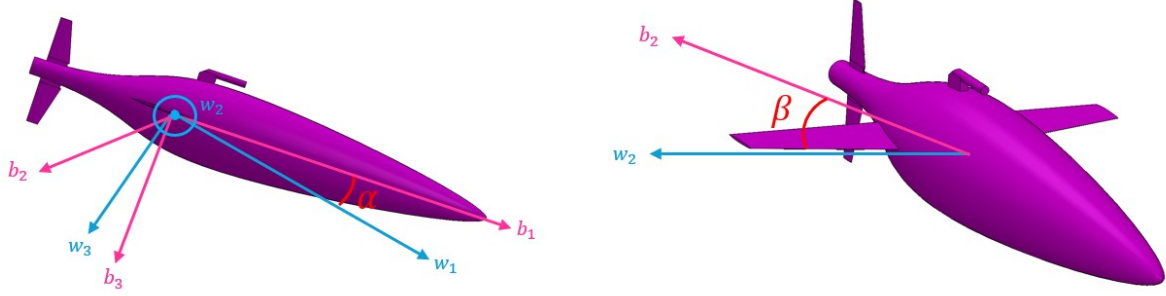


Fig. 7: Coordinate transformation from body to wind frame.

These forces are transformed into the body frame:

$$\mathbf{F}_{aero} = \mathbf{DCM}_{wb} \mathbf{F}_w \quad (18)$$

II.G. Buoyancy and Gravity

The fluid density ρ is approximated by the Seaglider's onboard sensors, where the range for the ocean is $\rho = [1020, 1030] \text{ (kg/m}^3\text{)}$.

Before the decision to use the Seaglider's output, the density gradient function was as follows:

$$\rho(z) = \begin{cases} 1.225, & z < 0 \\ \rho_0 + kz, & z \geq 0 \end{cases} \quad (19)$$

Where k is ρ as a function of temperature, pressure, and salinity.

Gravity expressed in the body frame is:

$$\mathbf{g}_b = \mathbf{DCM}_{gb} \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \quad (20)$$

Kyle's Approach:

$$m_{disp} = \rho V_{disp} \quad (21)$$

$$m_0 = m_s + m_{bat} - m_{disp} \quad (22)$$

$$\mathbf{B} = m_0 \mathbf{g}_b \quad (23)$$

Forces Team approach:

The Buoyant force on the Seaglider is [2]:

$$\mathbf{B} = \mathbf{g}_b \cdot (m_{total} - \rho \cdot \text{Volume}) \quad (24)$$

Where Volume in Equation 24 is:

$$\text{Volume} = \text{Vol}_{disp} \cdot e^{-(\kappa \cdot P - \tau \cdot (T - T_0))} \quad (25)$$

where

$$\text{Vol}_{disp} = \text{Vol}_{static} + \text{Vol}_{blad} \quad (26)$$

Taking the control input $V_{VBD,AD}$ and converting it to $V_{VBD,cc}$ to keep consistent units of cubic centimeters (cm^3) for volume:

$$V_{VBD,cc} = V_{VBD,AD} \cdot -0.2453 \frac{\text{cc}}{\text{AD count}} \quad (27)$$

Note that this is not being used as we take in $V_{VBD,cc}$ directly from the glider data.

Where Vol_{blad} is determined by

$$\text{Vol}_{blad} = -V_{VBD,cc} + 1426.7 \quad (28)$$

Final approach to buoyancy:

Using the same model as the Seaglider's on board buoyancy estimator which is as follows:

$$B = \frac{-m_{total} + \text{Volume} \cdot \frac{\rho(z)}{1000}}{10} \quad (29)$$

Equation 29 is the final equation to calculate buoyancy.

II.H. Mass and Inertia

The effective mass and inertia matrices are defined as:

$$\mathbf{M} = m_s \mathbf{I} + \mathbf{M}_f \quad (30)$$

$$\mathbf{J} = \mathbf{J}_s + \mathbf{J}_f \quad (31)$$

II.I. Battery Motion

The movable battery is modeled as a movable point mass.

The battery position in the body frame is:

$$\mathbf{r}_p = \begin{bmatrix} x_{bat} \\ r_{p,bat} \cdot \sin(\phi_{bat}) \\ r_{p,bat} \cdot \cos(\phi_{bat}) \end{bmatrix} \quad (32)$$

where $r_{p,bat}$ is the distance from the CB to the battery center of mass (CoM) in the (b_1, b_2) plane. It is found with the Law of Cosines:

$$r_{p,bat} = \sqrt{V_{CB}^2 + V_{CG,bat}^2 - 2V_{CB}V_{CG,bat}\cos(\phi_{bat})} \quad (33)$$

Where V_{CB} is the distance from the neutral axis to the CB in the vertical (b_3) direction, and $V_{CG,bat}$ is the distance from the neutral axis to the CoM of the high voltage battery in the b_3 direction which are obtained from SG 194 Trim and Balance sheet and SG 194 Mass and Inertia Matrix Calculation Document, respectively.

$[\mathbf{r}_p]_\times$ is skew-symmetric matrix of $\mathbf{r}_p$ is defined as:

$$[\mathbf{r}_p]_\times = \begin{bmatrix} 0 & -r_{p,z} & r_{p,y} \\ r_{p,z} & 0 & -r_{p,x} \\ -r_{p,y} & r_{p,x} & 0 \end{bmatrix} \quad (34)$$

II.J. CG Offset

The center of gravity of the static mass (the glider excluding the movable battery) was found in the SG 194 Trim and Balance Sheet to be 1.25 cm aft and 0.41cm below the CB. The vector going from the CB to the CG of the static mass, r_s , is defined in the body frame, in meters as:

$$r_s = [-0.0125 ; 0 ; 0.0041] \quad (35)$$

II.K. Moments

Torque coefficients are obtained from CFD and 3x3 wind tunnel testing:

$$C_{roll} = f_p(\alpha, \beta), \quad C_{pitch} = f_q(\alpha, \beta), \quad C_{yaw} = f_r(\alpha, \beta) \quad (36)$$

Hydrodynamic moments are given by:

$$\mathbf{T}_{hydro} = \begin{bmatrix} T_p \\ T_q \\ T_r \end{bmatrix} = \begin{bmatrix} qSb C_{roll} \\ qS\bar{c} C_{pitch} \\ qSb C_{yaw} \end{bmatrix} \quad (37)$$

II.L. Kinematics

In Seaglider dynamics, the rotational motion is described using the Euler angles:

$$\eta = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} \quad (38)$$

where ϕ is the roll angle, θ the pitch angle, and ψ the yaw angle. Sensors typically provide the Euler angular rates:

$$\dot{\eta} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \quad (39)$$

The body angular velocity vector $\mathbf{\Omega}$ describes angular velocities measured in the rotating body frame.

$$\mathbf{\Omega} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (40)$$

where p is roll rate about the b_1 axis, q is pitch rate about the b_2 axis, and r is the yaw rate about the body b_3 axis.

The matrix that transforms body angular velocity $\mathbf{\Omega}$ to Euler angular rates $\dot{\eta}$ is given by:

$$\mathbf{H}(\phi, \theta) = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \frac{\sin \phi}{\cos \theta} & \frac{\cos \phi}{\cos \theta} \end{bmatrix} \quad (41)$$

The relationship between Euler angle rates and angular velocity is:

$$\dot{\boldsymbol{\eta}} = \mathbf{H}\boldsymbol{\Omega} \quad (42)$$

The body angular velocity $\boldsymbol{\Omega}$ can be obtained by inverting the transformation matrix:

$$\boldsymbol{\Omega} = \mathbf{H}^{-1}\dot{\boldsymbol{\eta}} = \mathbf{H}^{-1} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (43)$$

The Euler angular accelerations can be obtained by taking the second derivative of Equation 42:

$$\ddot{\boldsymbol{\eta}} = \dot{\mathbf{H}}\boldsymbol{\Omega} + \mathbf{H}\dot{\boldsymbol{\Omega}} = \begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} \quad (44)$$

II.M. Equations of Motion

Translational acceleration with respect to body frame [1]:

$$\dot{\mathbf{v}}_{\mathbf{b}} = \mathbf{M}^{-1}\bar{\mathbf{F}} \quad (45)$$

where

$$\bar{\mathbf{F}} = (\mathbf{M}\mathbf{v}_{\mathbf{b}} + \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{B}z + \mathbf{F}_{\text{aero}} - \dot{\mathbf{P}}_p \quad (46)$$

Linear momentum of the moving battery pack in body frame [1]:

$$\mathbf{P}_p = \frac{\partial T}{\partial \dot{\mathbf{r}}_p} = m_{bat}(\mathbf{v}_{\mathbf{b}} + \boldsymbol{\Omega} \times \mathbf{r}_{\mathbf{p}} + \dot{\mathbf{r}}_{\mathbf{p}}) \quad (47)$$

where we assumed that $\dot{\mathbf{r}}_{\mathbf{p}} = 0$ because the changes in motion of the battery pack is modeled as instantaneous. Thus,

$$\mathbf{P}_p = m_{bat}\mathbf{v}_{\mathbf{b}} + m_{bat}(\boldsymbol{\Omega} \times \mathbf{r}_{\mathbf{p}}) \quad (48)$$

The rate of change of linear momentum of moving battery pack in body frame:

$$\dot{\mathbf{P}}_p = m_{bat}\dot{\mathbf{v}}_{\mathbf{b}} + m_{bat}(\dot{\boldsymbol{\Omega}} \times \mathbf{r}_{\mathbf{p}}) \quad (49)$$

Substituting Equation 45 and Equation 49 into Equation 46:

$$\mathbf{M}\dot{\mathbf{v}}_b = (\mathbf{M}\mathbf{v}_b + \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{B}z + \mathbf{F}_{aero} - m_{bat}\dot{\mathbf{v}}_b - m_{bat}(\dot{\boldsymbol{\Omega}} \times \mathbf{r}_p) \quad (50)$$

Rearranging to $F = ma$ form gives:

$$(\mathbf{M}\mathbf{v}_b + \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{B}z + \mathbf{F}_{aero} = \mathbf{M}\dot{\mathbf{v}}_b + m_{bat}\dot{\mathbf{v}}_b + m_{bat}(\dot{\boldsymbol{\Omega}} \times \mathbf{r}_p) \quad (51)$$

The translational dynamics are:

$$\mathbf{F}_{eq} = (\mathbf{M}\mathbf{v}_b + \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{B}z + \mathbf{F}_{aero} \quad (52)$$

Angular acceleration with respect to body frame [1] is:

$$\dot{\boldsymbol{\Omega}} = \mathbf{J}\bar{\mathbf{T}} \quad (53)$$

where

$$\bar{\mathbf{T}} = (\mathbf{J}\boldsymbol{\Omega} + [r_p]_{\times} \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{M}\mathbf{v}_b \times \mathbf{v}_b + m_{bat}g[r_p]_{\times} R^T z + \mathbf{T}_{hydro} - [r_p]_{\times} \dot{\mathbf{P}}_p \quad (54)$$

Substituting Equation 53 and Equation 49 into Equation 54:

$$\mathbf{J}\dot{\boldsymbol{\Omega}} = (\mathbf{J}\boldsymbol{\Omega} + [r_p]_{\times} \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{M}\mathbf{v}_b \times \mathbf{v}_b + m_{bat}g[r_p]_{\times} R^T z + \mathbf{T}_{hydro} - [r_p]_{\times} \dot{\mathbf{P}}_p \quad (55)$$

Rearranging to $T = J\dot{\Omega}$ form gives:

$$(\mathbf{J}\boldsymbol{\Omega} + [r_p]_{\times} \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{M}\mathbf{v}_b \times \mathbf{v}_b + m_{bat}g[r_p]_{\times} R^T z + \mathbf{T}_{hydro} = \mathbf{J}\dot{\boldsymbol{\Omega}} + [r_p]_{\times} m_{bat}\dot{\mathbf{v}}_b + [r_p]_{\times} m_{bat}(\dot{\boldsymbol{\Omega}} \times \mathbf{r}_p) \quad (56)$$

The rotational dynamics are:

$$\mathbf{T}_{eq} = (\mathbf{J}\boldsymbol{\Omega} + [r_p]_{\times} \mathbf{P}_p) \times \boldsymbol{\Omega} + \mathbf{M}\mathbf{v}_b \times \mathbf{v}_b + m_{bat}g[r_p]_{\times} R^T z + m_s[\mathbf{r}_s]_{\times} \mathbf{g}_b + \mathbf{T}_{hydro} \quad (57)$$

In matrix form, equivalent force and torque can be expressed as:

$$\begin{bmatrix} \mathbf{F}_{eq} \\ \mathbf{T}_{eq} \end{bmatrix} = \begin{bmatrix} \mathbf{M} + m_{bat}\mathbf{I} & -[\mathbf{r}_p]_{\times}m_{bat} - [\mathbf{r}_s]_{\times}m_s \\ m_{bat}[\mathbf{r}_p]_{\times} & \mathbf{J} - m_{bat}[\mathbf{r}_p]_{\times}[\mathbf{r}_p]_{\times} \end{bmatrix} \begin{bmatrix} \dot{\mathbf{v}}_b \\ \dot{\boldsymbol{\Omega}} \end{bmatrix} \quad (58)$$

The translational and rotational accelerations of the glider in body frame can be obtained from Equation 58.

$$\dot{\mathbf{v}}_b = \begin{bmatrix} \dot{u}_b \\ \dot{v}_b \\ \dot{w}_b \end{bmatrix} \quad (59)$$

$$\dot{\boldsymbol{\Omega}} = \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} \quad (60)$$

II.N. State Propagation

The acceleration in the glide-path frame is:

$$\dot{\mathbf{v}}_g = \mathbf{DCM}_{bg} (\dot{\mathbf{v}}_b + \boldsymbol{\Omega} \times \mathbf{v}_b) = \begin{bmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{bmatrix} \quad (61)$$

The final state derivative is:

$$\dot{\mathbf{X}} = \begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ heading \\ \dot{\phi} \\ \dot{\theta} \\ heading \\ \dot{u} \\ \dot{v} \\ \dot{w} \\ u \\ v \\ w \end{bmatrix} \quad (62)$$

This is fed through **ode45** function in Matlab to obtain the final state vector. ODE solver integrates time derivative of state over time to compute the next state of glider trajectory.



```
[t, X] = ode45(@(t,X) dynamicsV7(X, U, coefs, params), [0 dt], X0);
```

Fig. 8: Implementation of ode45 solver in Matlab

The final Matlab code that reflects this design is `dynamicsV7.m`

III. Sub-system Requirements

Dynamics Requirements	
ID	Requirement
DNR-1	Depth (z) of a dive must be modeled within 6.6% error of on-board recorded depth of a dive.
DNR-2	Vertical velocity (w) of a dive must be modeled within 3.3% error of on-board recorded vertical speed dz/dt of a dive.
DNR-3	Pitch of a dive must be modeled within 6.6% error of on-board recorded pitch angle of a dive.
DNR-4	Roll of a dive must be modeled within 6.6% error of on-board recorded roll angle of a dive.
DNR-5	Heading of a dive must be modeled within 6.6% error of on-board recorded heading angle of a dive.
DNR-6	Pitch rate of a dive must be modeled within 3.3% error of on-board recorded pitch rate of a dive.
DNR-7	Roll rate of a dive must be modeled within 3.3% error of on-board recorded roll rate of a dive.
DNR-8	Heading rate of a dive must be modeled within 3.3% error of on-board recorded heading rate of a dive.

Table 2: Dynamics Modeler Requirements

Forces Requirements	
ID	Requirement
FMR-1	The Forces Modeler shall compute the lift force within 3.30% error of the lift force from the existing nominal planar flight hydrodynamic model.
FMR-2	The Forces Modeler shall compute the drag force within 3.30% error of the drag force from the existing nominal planar flight hydrodynamic model.
FMR-3	The Forces Modeler shall compute buoyant force within 3.30% error of the buoyancy estimator on board the Seaglider.

Table 3: Forces Modeler Requirements

IV. Error Budget Allocation for Dynamics and Force Requirements

The total allowable error from SR-3 system requirement is given as:

$$E_{\text{tot}} = 39.9\% \quad (63)$$

This error budget is only distributed across state variables that the diagnoser will be taking in:

$$\phi \quad \theta \quad \psi \quad z$$

IV.A. Error Percent Allocation for States (used in PDR and CDR)

The total error is the sum of errors of states used by the Diagnoser:

$$E_{\text{tot}} = E_{\theta} + E_{\phi} + E_{\psi} + E_{\dot{\theta}} + E_{\dot{\phi}} + E_{\dot{\psi}} + E_z + E_{\dot{z}} \quad (64)$$

The error of position is derived from the errors from steps taken to derive it. It is assumed to have equal error distribution of error among velocities and acceleration.

$$E_x = E_v + E_a = 2E_v \quad (65)$$

Applying this to the states in the Equation 71,

$$E_{\dot{\theta}} = \frac{1}{2}E_{\theta} \quad (66)$$

$$E_{\dot{\phi}} = \frac{1}{2}E_{\phi} \quad (67)$$

$$E_{\dot{\psi}} = \frac{1}{2}E_{\psi} \quad (68)$$

$$E_{\dot{z}} = \frac{1}{2}E_z \quad (69)$$

Substituting these into Equation 71:

$$E_{\text{tot}} = E_{\theta} + E_{\phi} + E_{\psi} + \frac{1}{2}E_{\theta} + \frac{1}{2}E_{\phi} + \frac{1}{2}E_{\psi} + E_z + \frac{1}{2}E_z = \frac{3}{2}(E_{\theta} + E_{\phi} + E_{\psi} + E_z) \quad (70)$$

$$\frac{3}{2}(E_{\theta} + E_{\phi} + E_{\psi} + E_z) = 39.9\% \quad (71)$$

Solving for equal state error:

$$E_{\theta} = E_{\phi} = E_p = E_z = e \quad (72)$$

$$39.9\% = \frac{3}{2}(4e) \quad (73)$$

$$e = 6.6\% \quad (74)$$

Then, the error is equally distributed across these for variables:

$$E_{\theta} = 6.6\% \quad E_{\phi} = 6.6\% \quad E_{\psi} = 6.6\% \quad E_z = 6.6\%$$

The sum of errors of all the angles:

$$E_{\text{orientation}} = 6.6\% \cdot 3 = 19.80\%$$

The error for velocities can then be solved for using Equation 66, Equation 67, Equation 68, and Equation 69:

$$E_{\dot{\theta}} = 3.3\% \quad E_{\dot{\phi}} = 3.3\% \quad E_{\dot{\psi}} = 3.3\% \quad E_{\dot{z}} = 3.3\%$$

The sum of errors of all the angle rates:

$$E_{\text{orientation rates}} = 3.3\% \cdot 3 = 9.9\%$$

Since error of velocity and error of accelerations are equal due to equal distribution in

Equation 65:

$$E_{\ddot{\theta}} = 3.3\% \quad E_{\ddot{\phi}} = 3.3\% \quad E_{\ddot{\psi}} = 3.3\% \quad E_{\ddot{z}} = 3.3\%$$

The error of acceleration is derived from Newton's second law. The error for masses is approximate to be zero.

$$E_a = \overset{0}{E}_m + E_F \quad (75)$$

$$E_a = E_F \quad (76)$$

In this case, $\ddot{z}$ is derived using B , L , D , and Y . By the logic of Equation 76 :

$$E_B = 3.3\% \quad E_L = 3.3\% \quad E_D = 3.3\% \quad E_Y = 3.3\%$$

Summary of Error Budget

Quantity	Assigned Error
Position (θ, ϕ, p, z)	6.6%
Velocity	3.3%
Acceleration	3.3%
Force/Torque	3.3%

This analysis represents a first-order error budget allocation.

Assumptions:

- Errors propagate through derivatives (position $\rightarrow$ velocity $\rightarrow$ acceleration)
- Force uncertainty is proportional to acceleration uncertainty via Newton's Second Law

IV.B. Error Magnitude Calculations (used in TRR, AR)

NOTE: Error calculations were revised to reflect error magnitudes rather than error percentages. This approach was selected to keep the allowable error consistent throughout the dive profile rather than allowing it to vary with the output value (for ex: we do not want error allowed to increase as depth increases).

Allowed error for each requirement depends on each dive profile.

Var = output from the dynamics code (depth, vertical velocity, roll, pitch, heading, etc.)

$$\text{Range} = \text{MAX}(\text{Var}_{\text{mission}}) - \text{MIN}(\text{Var}_{\text{mission}})$$

$$\text{Allowed Error: } E_{\text{allow}} = (\text{MAX}(\text{Var}_{\text{mission}}) - \text{MIN}(\text{Var}_{\text{mission}})) \cdot 6.6\% \text{ or } 3.3\%$$

$$\text{Actual Error: } E_{\text{actual}} = |\text{Var}_{\text{mission}} - \text{Var}_{\text{simulation}}|$$

- Dynamics Example in TRR for depth:

4/17/2026 Edmonds Field Test Dive 5:

$$E_{\text{allow},z} = |\text{MAX}(\mathbf{z}_{\text{mission}}) - \text{MIN}(\mathbf{z}_{\text{mission}})| \cdot 6.6\% = |88.15 - 0.87| \cdot 6.6\% = 5.7605$$

$$E_{\text{allow},z} = \pm 5.7605 \text{ m}$$

- Forces Example in TRR for lift force:

$$E_{\text{allow},L} = |\text{MAX}(L_{HDM}) - \text{MIN}(L_{HDM})| \cdot 0.033 = |0.5217 - (-0.5217)| \cdot 0.033 = \pm 0.0344N$$

NOTE: For calculating the error for the drag force, instead of using the minimum and maximum, the average of all the drag forces was taken and then multiplied by 0.033. The reason for doing so is the drag force from the HDM is nearly constant, which would give error magnitudes of 8e-4.

V. References

1. Leonard, Naomi E. and Graver, Joshua G. (2001). *Model-Based Feedback Control of Autonomous Underwater Gliders*
2. Bennett, James S., Frederick R. Stahr, Charles C. Eriksen, Martin C. Renken, Wendy E. Snyder, and Lora J. Van Uffelen. *Assessing Seaglider Model-Based Position Accuracy on an Acoustic Tracking Range*. Journal of Atmospheric and Oceanic Technology 38.6 (2021): 1111-1123. <https://doi.org/10.1175/JTECH-D-20-0091.1> Web.

VI. Appendix A: Tables and Figures

Variable	Description	Units	Source	Data Name
Coefficients				
coefs.alphas (α)	Angle of attack	deg	Wind tunnel testing and CFD 1xM vector	
coefs.betas (β)	Side slip angle	deg	Wind tunnel testing and CFD 1xN vector	
coefs.CLs (C_L)	Lift coefficient		Wind tunnel testing and CFD MxN matrix	
coefs.CDs (C_D)	Drag coefficient		Wind tunnel testing and CFD MxN matrix	
coefs.CYs (C_Y)	Side force coefficient		Wind tunnel testing and CFD MxN matrix	
coefs.Croll (C_{roll})	Roll moment coefficient		Wind tunnel testing and CFD MxN matrix	
coefs.Cpitch (C_{pitch})	Pitch moment coefficient		Wind tunnel testing and CFD MxN matrix	
coefs.Cyaw (C_{yaw})	Yaw moment coefficient		Wind tunnel testing and CFD MxN matrix	

Table 4: Coefficients

Parameter	Code Name	Description	Units	Value/Range	Source
S	<code>params.S</code>	Surface area of glider in XY plane	m ²	0.486	CAD model
$\bar{c}$	<code>params.cbar</code>	Mean aerodynamic chord	m	0.173	CAD model
b	<code>params.b</code>	Wing span	m	1	CAD model
$\psi_{desired}$	<code>params.heading.desired</code>	Desired heading angle with respect to magnetic north	deg or rad	Varies with dive	Log files / <code>seaglider.pub</code>
V_{CB}	<code>params.rCB</code>	Distance from neutral axis to center of buoyancy in the b_3 direction	m	−0.00362	SG 194 WA COAST-230ct-2013 data sheet
$V_{CG,bat}$	<code>params.rCGbat</code>	Distance from neutral axis to battery center of mass in the b_3 direction	m	−0.01082	SG 194 Mass and Inertia Matrix Calculation Document
m_{bat}	<code>params.mbat</code>	Battery mass	kg	11.636	Ballasting data sheet
V_{static}	<code>params.Vstatic</code>	Static displaced volume of glider excluding bladder	m ³	0.0552	SG 194 Trim and Balance sheet, where total volume is 57290 cc when VBD is at 2,640 AD count.
$\mathbf{M}_f$	<code>params.Mf</code>	Added mass matrix	kg	$\begin{bmatrix} 3.310 & 0 & 0 \\ 0 & 72.005 & 0 \\ 0 & 0 & 72.005 \end{bmatrix}$	SG 194 Mass and Inertia Matrix Calculation Document
$\mathbf{J}_f$	<code>params.Jf</code>	Added inertia matrix	kg·m ²	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 10.062 & 0 \\ 0 & 0 & 10.062 \end{bmatrix}$	SG 194 Mass and Inertia Matrix Calculation Document
$\mathbf{J}_s$	<code>params.Js</code>	Stationary inertia matrix	kg·m ²	$\begin{bmatrix} 0.1009 & -0.0604 & 0.1086 \\ -0.0604 & 8.1926 & -0.0015 \\ 0.1086 & -0.0015 & 8.1437 \end{bmatrix}$	SG 194 Mass and Inertia Matrix Calculation Document
m_s	<code>params.Ms</code>	Stationary glider mass	kg	58.852	SG 194 WA COAST-230ct-2013 data sheet
ρ	<code>params.rho</code>	Water density	kg/m ³		Log files
P	<code>params.P</code>	Pressure	dbar		Log files
T	<code>params.T</code>	Water temperature	°C		Log files
T_0	<code>params.ambtemp</code>	Ambient temperature of water surface	°C		Log files
$salt$	<code>params.salt</code>	Salinity	PSU		Profile file

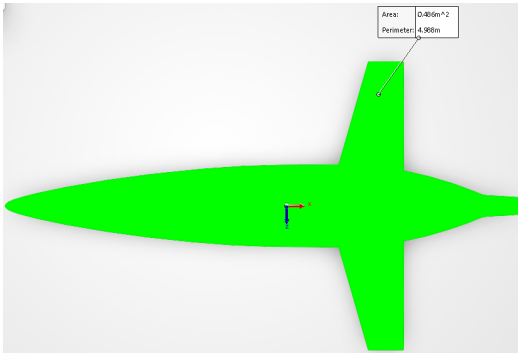


Fig. 9: Surface area S

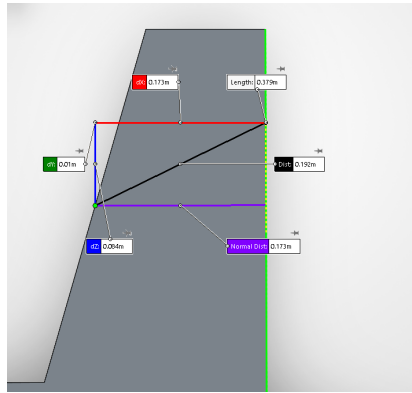


Fig. 10: Mean Aerodynamic Chord $\bar{c}$

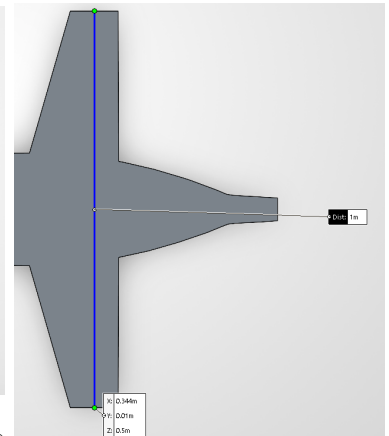


Fig. 11: Wing span b

Fig. 12: Properties for Aerodynamic Surfaces

			Qty	Weight grams	Total Weight grams
2	Assy., Mass shifter	49889			
	Scale Weight	14166.0	gm	14166.0	2530.4
	Moveable Weight				
	Battery Enclosure	56759	1	478.0	478.0
	Brass weight		1	1372.6	1372.6
	HV Battery	52238	1	9785.0	9785.0

Fig. 13: Movable battery mass

Variable	Description	Units	Source	Data Name
State Inputs				
$\dot{\phi}$	roll rate	deg/s	Calculated	
$\dot{\theta}$	pitch rate	deg/s	Calculated	
$\dot{heading}$	Current heading rate	deg/s	Calculated	
ϕ	roll angle	deg	ENG	rollAng
θ	pitch angle	deg	ENG	pitchAng
$heading$	Current heading angle of glider with respect to magnetic north	deg	ENG	head
u	horizontal velocity glide path frame	m/s	initial/previous state	
v	lateral velocity glide path frame	m/s	initial/previous state	
w	vertical velocity glide path frame	m/s	initial/previous state	
x	x distance in glide path frame	m	initial/previous state	
y	y distance in glide path frame	m	initial/previous state	
z	z distance/depth in glide path frame	m	initial/previous state	
Control Inputs				
x_{bat}	battery position in b_1 axis (aft positive)	m	ENG	pitchCtl
ϕ_{bat}	battery rotation in yz-plane	deg	ENG	rollCtl
$V_{VBD,cc}$	volume of oil in VBD	cc	ENG	vbdCC

Table 6: Inputs

VII. Appendix B: Final Code (dynamicsv7.m)

Table of Contents

.....	1
Unpacking Inputs	1
Translational velocity, alpha, beta	3
Mass and Inertia	3
Forces & Coefs	3
Moments (torques)	5
Momentum (finally)	5
Dx output vector	6

```
% Seaglider dynamics function for digital twin via momentum
% To be put through your favorite ODE solver
% Author: Kyle Wittcoff
% Date: June 2026
function X_dyn = dynamicsV7(X,U,coefs,params)
```

Unpacking Inputs

```
% Avoiding divide by zero errors

% Unpacking state vector X
phidot = X(1) * pi/180; % roll rate (deg/s to rad/s)
thetadot = X(2) * pi/180; % pitch rate (deg/s to rad/s)
headingdot = X(3) * pi/180; % yaw rate (deg/s to rad/s)

phi = X(4) * pi/180; % roll angle (deg to rad)
theta = X(5) * pi/180; % pitch angle (deg to rad)
heading = X(6) * pi/180; % heading angle (deg to rad)

u_i = X(7); % horizontal velocity wrt current heading
v_i = X(8); % lateral velocity wrt current heading
w = X(9); % vertical velocity in glide path frame

x = X(10); % x distance in Glide Path frame
y = X(11); % y distance in Glide Path frame
z = X(12); % z distance (depth) in Glide Path frame

% Unpacking control input
pitch_ctl = U(1); % battery pos down x axis (in body frame, aft positive)
phi_bat = U(2); % battery roll angle
VBD_cc = U(3); % oil count in the VBD pump in AD

% Battery CG longitudinal location, measured from the CB (body +x = nose)
X_bat0 = 0.0509; % neutral position, from 194 trim sheet
x_bat = X_bat0 - pitch_ctl ; % battery location relative to CB, m

% Unpacking parameters

% Target information
```

```

heading_desired = deg2rad(params.heading_desired); % desired heading for dive

% Aerodynamic surfaces properties
S = params.S; % wing surface area
cbar = params.cbar; % wing MAC
b = params.b; % wing span

% Glider properties

% Whole glider
Vol_static = params.Vstatic; % displaced volume w/o VBD
Mf = params.Mf; % 3x3 added mass matrix, should be changed later?
Jf = params.Jf; % 3x3 added mass inertia matrix, should be changed later?
ms = params.Ms; % stationary mass
Js = params.Js; % 3x3 stationary mass inertia matrix (referenced to the CB)
VCB = 0.00362; % Vertical CB location relative to neutral axis, in m for SG
194

% Stationary-mass CG offset from the CB, body frame, from 194 trim sheet
r_s = [-0.0125; 0; 0.0041];

% Battery
VCG_bat = 0.01082; % Battery vertical cg location relative to neutral axis,
in m for SG 194
mbat = params.mbat; % battery pack mass

% Ocean properties
rho = params.rho; % density of ocean from CT

% Unpacking coefficients
alphas = coefs.alphas; % 1 x M vector of alphas used in wind tunnel test
betas = coefs.betas; % 1 x N vector of betas used in wind tunnel test
CLs = coefs.CLs; % M x N array of lift coef values from wind tunnel test
Cds = coefs.CDs; % M x N array of drag coef values from wind tunnel test
Cys = coefs.CYs; % M x N array of side coef values from wind tunnel test
Crolls= coefs.Croll;% M x N array of roll coef values from wind tunnel test
Cpitches=coefs.Cpitch;%MxN array of pitch coef values from wind tunnel test
Cyaws= coefs.Cyaw;% M x N array of yaw coef values from wind tunnel test

% Yaw angle
psi = heading - heading_desired;

% Battery position rp in body frame (relative to the CB).
% rp_z uses (VCG_bat*cos - VCB): at phi_bat=0 this gives the battery CG
% below the CB, matching the measured 0.73 cm offset for SG 194
rp = [x_bat;
      abs(VCG_bat) * sind(phi_bat);
      abs(VCG_bat) * cosd(phi_bat) - VCB];
skew = @(v) [0 -v(3) v(2); v(3) 0 -v(1); -v(2) v(1) 0];
rpx = skew(rp); % 3x3 cross product matrix for rp
rsx = skew(r_s); % 3x3 cross product matrix for r_s

```

Not enough input arguments.

Error in dynamicsV7 (line 11)
phidot = X(1) * pi/180; % roll rate (deg/s to rad/s)
^^^^

Translational velocity, alpha, beta

```
% Horizontal velocity in glide path frame.
% (u_i,v_i) are referenced to the current heading; they differ from the
% glide-path-frame (u,v) by a pure rotation about the vertical axis through
% the heading error psi
u = u_i * cos(psi) - v_i * sin(psi);
v = u_i * sin(psi) + v_i * cos(psi);

% DCM for body frame to glide path frame
DCM_gb = angle2dcm(psi, theta, phi); % DCM glide to body
DCM_bg = DCM_gb.'; % DCM body to glide (it's just an inverse :) )

% Glide path and body frame velocity
V_g = [u;v;w]; % glide path frame velocity
V_b = DCM_gb * V_g; % body frame velocity
ub = V_b(1);
vb = V_b(2);
wb = V_b(3);
% Note: We CAN implement the water currents in this part. Some frame
% silliness may be required.

% Flight path angles
V = sqrt(u^2 + v^2 + w^2); % Speed, norm function is for cowards
if V > 1e-6
    alpha = atan2(wb, ub) * 180/pi; % deg, body-frame angle of attack
    beta = asin(vb / V) * 180/pi; % deg, body-frame sideslip
else
    alpha = 0; beta = 0;
end
```

Mass and Inertia

```
% Effective translational and rotational inertia matrices
M = ms*eye(3) + Mf; % masses
J = Js + Jf ; % inertias
```

Forces & Coefs

```
% Dynamic Pressure
q = 0.5 * rho * V^2;

% Clamping aoa and sideslip so coefs don't blow up
alpha_clamp = min(max(alpha, min(alphas)), max(alphas)); %give em the clamps
```

```

% Sideslip: the coefficient tables are tabulated for beta >= 0 only.
% For negative sideslip, use the lateral symmetry of the (symmetric)
% vehicle: look up at |beta|, then apply the correct parity --
%   even in beta (CL, CD, Cpitch): coefficient unchanged
%   odd  in beta (CY, Croll, Cyaw): coefficient negated
% This replaces the old scheme that clamped all negative beta to 0.
beta_abs = min(max(abs(beta), min(betas)), max(betas));
beta_sign = 1;
if beta < 0
    beta_sign = -1;
end

% Coefs from lookup table
% Even-in-beta coefficients (use |beta| directly)
CL      = interp2(betas, alphas, CLs,      beta_abs, alpha_clamp, 'linear');
CD      = interp2(betas, alphas, Cds,      beta_abs, alpha_clamp, 'linear');
Cpitch  = interp2(betas, alphas, Cpitches, beta_abs, alpha_clamp, 'linear');
% Odd-in-beta coefficients (negate for negative sideslip)
CY      = beta_sign * interp2(betas, alphas, Cys,
    beta_abs, alpha_clamp, 'linear');
Croll   = beta_sign *
    interp2(betas, alphas, Crolls, beta_abs, alpha_clamp, 'linear');
Cyaw    = beta_sign * interp2(betas, alphas, Cyaws,
    beta_abs, alpha_clamp, 'linear');

% Hydrodynamic forces in body frame
L = q * CL * S; % lift
D = q * CD * S; % drag
Y = q * CY * S; % side
F_hydro = [-D; Y; -L];

% Buoyancy Force -- MAK

% Total displaced volume & mass
Vol_blad = (VBD_cc + 1426.7) * 1e-6; % oil volume in bladder m^3
Vol_disp = Vol_blad + Vol_static; % total volume displaced
Volume = Vol_disp; %* exp(-(Kappa * P - tau * (T - T0))); % -- MAK
m_total = ms + mbat; % total mass of the seaglider (ms will change with
damage cases)

% Gravity
g = 9.81; % m/s^2

% Gravity in body frame
g_b = DCM_gb * [0; 0; g];

% buoyancy equation
B = g_b * (m_total - rho * Volume) ;

```

Moments (torques)

```
% Hydrodynamic moments (torques)
Mp = q * Croll * b * S; % roll moment
Mq = q * Cpitch * cbar * S; % pitch moment
Mr = q * Cyaw * b * S; % yaw moment
Torques = [Mp; Mq; Mr];

% Euler-rate vector in rad/s
eta_dot = [phidot; thetadot; headingdot]; %sensors give us euler angles
                                              %we'll convert for RB dynamics

% Kinematics matrix: eta_dot = H * omega, where omega = [p;q;r]
H = [1 sin(phi)*tan(theta) cos(phi)*tan(theta);
     0 cos(phi) -sin(phi);
     0 sin(phi)/cos(theta) cos(phi)/cos(theta)];

% Time derivative of H
Hdot = [0, cos(phi)*phidot*tan(theta) + sin(phi)*sec(theta)^2*thetadot, ...
        -sin(phi)*phidot*tan(theta) + cos(phi)*sec(theta)^2*thetadot;
        0, -sin(phi)*phidot, ...
        -cos(phi)*phidot;
        0, cos(phi)*phidot*sec(theta) +
sin(phi)*sec(theta)*tan(theta)*thetadot, ...
        -sin(phi)*phidot*sec(theta) +
cos(phi)*sec(theta)*tan(theta)*thetadot];

% Body angular velocity
Omega = H \ eta_dot; % Omega = [p; q; r] body rates, rad/s
```

Momentum (finally)

```
% Battery translational momentum
Pp = mbat * (V_b + cross(Omega, rp));

% Internal-mass coupling vector for the 6x6 mass matrix.
% L&G's general off-diagonal block is (mbat*rp_hat + ms*r_s_hat); the paper
% drops the ms*r_s term only because it specializes r_s = 0.
coupx = mbat*rp_x + ms*rs_x;

% Starting with vdot = M^-1 Fbar, see algebra in notes
% Feq = cross(M*V_b + Pp, Omega) + m0*g_b + F_hydro;
Feq = cross(M*V_b + Pp, Omega) + B + F_hydro;
veq = [M + mbat*eye(3), -coupx];

% Starting with Omegadot = J^-1 Tbar, see algebra in notes
% Gravity moment now includes BOTH internal masses: the battery
% (mbat at rp) and the stationary mass (ms at r_s)
Teq = cross(J*Omega + cross(rp, Pp), Omega) ...
      + cross(M*V_b + Pp, V_b) ...
      + mbat * cross(rp, g_b) ...
```

```

    + ms * cross(r_s, g_b) ...
    + Torques;
Omegaeq = [coupx, J - mbat*rp*rp];

```

```

% Accelerations from equation in notes
accels = [veq ; Omegaeq] \ [Feq ; Teq];
Vdot_b = accels(1:3); % translational
Omegadot = accels(4:6); % angular

```

Dx output vector

```

% Body angular accel to Euler angular accel
eta_Ddot = Hdot * Omega + H * Omegadot ; % chain rule :)

% Body accel to glide accel
Vdot_g = DCM_bg * ( Vdot_b + cross(Omega,V_b) );
udot = Vdot_g(1); % glide-path-frame accelerations
vdot = Vdot_g(2);
wdot = Vdot_g(3);

% Converting glide-path accelerations (udot,vdot) to heading-referenced
derivatives
% The state stores u_i, v_i (heading-referenced velocities), which relate
% to the glide-path velocities u, v by a pure rotation about psi:
% [udot; vdot] = Rdot*[u_i;v_i] + R*[u_idot;v_idot]
% ==> [u_idot; v_idot] = R \ ([udot;vdot] - Rdot*[u_i;v_i])
R = [cos(psi), -sin(psi);
     sin(psi),  cos(psi)];

% psi = heading - heading_desired (heading_desired constant)
% ==> psidot = headingdot
psidot = headingdot; % rad/s
Rdot = psidot * [-sin(psi), -cos(psi);
                cos(psi), -sin(psi)];

% Horizontal and lateral accelerations
ui_vi_dot = R \ ([udot; vdot] - Rdot*[u_i; v_i]);
u_idot = ui_vi_dot(1); % heading-referenced horizontal acceleration
v_idot = ui_vi_dot(2); % heading-referenced lateral acceleration

% Converting rad/s back to deg/s
phiDdot = eta_Ddot(1) * 180/pi;
thetaDdot = eta_Ddot(2) * 180/pi;
headingDdot = eta_Ddot(3) * 180/pi;
phidot = phidot * 180/pi;
thetadot = thetadot * 180/pi;
headingdot = headingdot * 180/pi;

% Output
X_dyn = [phiDdot; thetaDdot; headingDdot; phidot; thetadot; headingdot; ...
        u_idot; v_idot; wdot; u_i; v_i; w];

end

```

Published with MATLAB® R2025b